AWS SAA-C03 Study Notes: Containers on AWS

1. Docker Introduction

For the exam, you don't need to be a Docker expert, but you must understand the terminology to interpret architecture questions.

- **Docker:** A platform to develop, ship, and run applications in isolated environments called containers.
- **Container:** A lightweight, standalone, executable package that includes everything needed to run a piece of software (code, runtime, system tools, libraries).
- **Dockerfile:** A text document that contains all the commands to assemble an image.
- **Docker Image:** A read-only template with instructions for creating a Docker container.
- **Registry:** A place to store images (e.g., Docker Hub, **Amazon ECR**).
- **Key Exam Concept:** Containers solve the "it works on my machine" problem and provide portability across different computing environments (Dev -> Test -> Prod).

2. Amazon ECS (Elastic Container Service)

ECS is AWS's proprietary container orchestration tool. It is highly integrated with other AWS services.

Core Components

1. **Cluster:** A logical grouping of tasks or services. It can run on EC2 instances or Fargate (serverless).
2. **Task Definition:** A blueprint (JSON file) that describes the container(s) (e.g., Image URI, CPU/Memory, Ports, Environment Variables). Similar to a Docker Compose file.
3. **Task:** A running instance of a Task Definition.
4. **Service:** Ensures that a specified number of Tasks are constantly running. Handles replacing failed tasks and registering them with Load Balancers.

Compute Options & Launch Types (CRITICAL for Exam)

There are now three primary ways to host your tasks, ranging from fully serverless to fully self-managed.

Feature	Fargate (Serverless)	ECS Managed Instances (Hybrid)	EC2 Launch Type (Self-Managed)
Management	AWS manages everything. No OS access.	AWS provisions & patches EC2s. You can access the OS.	You manage the EC2s (ASG, patching, OS updates).

Feature	Fargate (Serverless)	ECS Managed Instances (Hybrid)	EC2 Launch Type (Self-Managed)
Infrastructure	Serverless.	Instances in your account, managed by AWS.	Instances in your account, managed by you.
Capabilities	Limited (No GPU, specific storage limits).	**Full EC2 features** (GPUs, Local NVMe, Spot).	**Full EC2 features** (GPUs, Local NVMe, Spot).
Use Case	Low ops overhead, batch jobs, general web apps.	AI/ML (GPUs) or high-performance apps requiring specific instances but **no ops overhead**.	Legacy compliance, granular control, or specific cost optimization strategies.

3. Amazon ECS - Auto Scaling

Scaling in ECS happens at two levels: The Service (Application) and the Cluster (Infrastructure).

Service Auto Scaling (Application Level)

Uses **Application Auto Scaling** to increase/decrease the number of Tasks.

- **Target Tracking:** "Keep CPU utilization at 70%." (Most common).
- **Step Scaling:** "If alarm breaches X, add 2 tasks."
- **Scheduled Scaling:** "At 8 AM, scale to 10 tasks."

Cluster Auto Scaling (Infrastructure Level)

- **Capacity Providers:** The link between ECS and the Auto Scaling Group (ASG).
- **Scenario:** If you have 2 EC2 instances and they are full, and you try to launch a new Task, it will fail.
- **Solution:** The **ECS Capacity Provider** detects that a task is in a `PROVISIONING` state (waiting for resources) and triggers the ASG to launch new EC2 instances automatically.

4. Amazon ECS - Solutions Architectures

Pattern A: Microservices behind ALB

- **Architecture:** ALB (Public Subnet) -> ECS Service (Private Subnet).
- **Dynamic Port Mapping:** When using **EC2 Launch Type**, you can set the host port to 0. The ALB automatically detects which random port the container was assigned and routes traffic there. This allows multiple copies of the same container on a single EC2 instance.

Pattern B: Event-Driven Tasks

- **Architecture:** S3 Event -> EventBridge -> ECS Task.
- **Use Case:** An image is uploaded to S3; an ECS task spins up to resize it and then shuts down.
- **Exam Tip:** Use **Fargate** here. You don't want to pay for an EC2 instance running 24/7 waiting for an event.

Pattern C: Persistent Storage

- **Data Persistence:** Containers are ephemeral (data is lost when they stop).
- **Solution:** Mount **Amazon EFS** (Elastic File System) to the ECS tasks.
- **Why EFS?** It is regional and can be mounted by multiple containers across different AZs simultaneously. (EBS cannot do this).

5. Amazon ECR (Elastic Container Registry)

A managed Docker registry service. Stores, manages, and deploys container images.

- **Private vs. Public:** You can host private repositories (internal use) or public ones (Amazon ECR Public Gallery).
- **Security Scanning:** "Basic Scanning" (on push) or "Enhanced Scanning" (continuous using Amazon Inspector) to find vulnerabilities in images.
- **Lifecycle Policies:** Automate the cleanup of old images (e.g., "Expire images older than 30 days" or "Keep only the last 10 untagged images").
- **Replication:** Supports **Cross-Region Replication (CRR)** and Cross-Account Replication. Useful for disaster recovery or reducing latency for global deployments.

6. Amazon EKS (Elastic Kubernetes Service)

Managed Kubernetes on AWS. Choose this if you are already using Kubernetes or need open-source compatibility.

- **Architecture:**
  - **Control Plane:** Managed by AWS (highly available, runs across multiple AZs).
  - **Data Plane (Nodes):** Where your pods run. Can be:
    1. **Managed Node Groups:** AWS manages the EC2 provisioning/lifecycle.
    2. **Self-Managed Nodes:** You manage EC2s manually.
    3. **AWS Fargate:** Serverless pods (no EC2 management).
- **ECS vs. EKS:**
  - **ECS:** Simpler, AWS-native, easier integration with IAM/ALB.
  - **EKS:** steeper learning curve, portable to other clouds (Azure/GCP), huge ecosystem (Helm, Istio).

7. AWS App Runner

The easiest way to run a containerized web application or API. Fully managed.

- **Concept:** "Here is my source code" or "Here is my container image" -> AWS gives you a HTTPS URL.
- **Under the hood:** It builds the image, deploys the load balancer, sets up auto-scaling, and manages encryption certificates automatically.
- **Use Cases:** Request-Response web services (APIs, frontends).
- **NOT for:** Long-running batch jobs (use ECS/Batch) or stateful workloads requiring complex networking.

8. AWS App2Container (A2C)

A migration tool for modernizing legacy applications.

- **Function:** CLI tool that runs on your server (On-Prem or EC2). It analyzes applications and containerizes them.
- **Supported Applications:**
  - **Java** (Linux).
  - **.NET** (Windows IIS).
- **Workflow:**
  1. **Discover:** Identifies running apps (IIS websites, Java processes).
  2. **Analyze:** Creates an analysis report.
  3. **Extract:** Generates a Dockerfile and builds the container image.
  4. **Deploy:** Pushes the image to **ECR** and generates deployment artifacts (CloudFormation/YAML) for **ECS** or **EKS**.
- **Exam Key:** If a question asks how to **"move a legacy Java/.NET app to containers with minimal effort,"** the answer is **App2Container**.

Summary Cheat Sheet for SAA-C03

Scenario	Service
Simple container orchestration, deep AWS integration	**ECS**
Kubernetes, open-source compatibility, multi-cloud	**EKS**
Easiest way to deploy a containerized web app URL	**App Runner**
Run containers without managing servers	**Fargate**
Run containers with GPUs but AWS manages the OS	**ECS Managed Instances**
Store and scan Docker images	**ECR**
Migrate .NET/Java to Containers	**App2Container**
Containers need persistent storage across AZs	**ECS + EFS**